

Front-loaded validation

A review of the training plan | Sarel Ye & Sary | 23 September 2026

Your strategy is reasonable for this pilot. Validate frequently while learning whether the training process works, then reduce frequency when nearby checks are likely to provide similar information. This is a useful hypothesis about information value, not a guarantee that validation becomes steadily less useful.

1. Check steps and intervals are different

The sequence "10, 20, 30, 50, 100" can describe either absolute optimizer steps or the gaps between checks:

Interpretation	Actual steps at validation
Absolute steps, then every 100	10, 20, 30, 50, 100, 200, 300...
Increasing intervals between checks	10, 30, 60, 110, 210, 310...

The document currently specifies **absolute steps 0, 10, 20, 50, 100, 200...**, omitting 30. Step 0 evaluates the untrained model and provides a useful baseline. The intended interpretation should be explicit in the plan.

2. What makes a validation check valuable?

Validation costs computation without directly updating parameters. Its benefit comes from informing decisions: detecting problems, selecting a checkpoint, or stopping training.

Early in a run: frequent checks can reveal whether held-out error is improving and whether errors or scaling look suspicious.

During stable progress: closely spaced checks often add little information, so spacing them out can save computation.

Later: validation can become valuable again when overfitting begins, training becomes unstable, or accuracy approaches the acceptance target.

The diminishing quantity is often **the additional information from another nearby check**, rather than the usefulness of validation overall. A slowly changing curve supports longer intervals, but training need not remain predictable.

Cost, trade-offs, and a practical choice

3. What computation does the schedule save?

Moving from validation every 10 updates to every 100 reduces validation work by approximately **90% during that phase**. It does not reduce total training work by 90%.

If one optimizer update takes **T** seconds, one complete validation takes **V** seconds, and the validation interval is **k** updates, the validation share of runtime is approximately:

$$\text{Validation share} = V / (kT + V)$$

Measure training time and validation time separately in the pilot. Reduced runtime is evidence of compute savings; actual electrical-energy savings require power measurements.

Compared with validating every 100 updates from the start, front-loading **adds a small cost** to obtain earlier feedback. The savings therefore depend on the schedule used as the comparison.

4. Recommendation for this experiment

Keep this simple schedule for the pilot, including **step 0 and the final step**, and use identical check steps across architectures. Treat the final interval of 100 as a starting setting to evaluate, not an established optimum.

The main trade-off is observation resolution. After switching to 100-update intervals, a better intermediate checkpoint may go unobserved. Report the **"first observed passing check"**, rather than the exact update where the model first became accurate.

If early stopping is added later, express patience in optimizer updates so changing validation intervals does not silently change its meaning. Stopping decisions can still occur only at scheduled validation checks.

5. A related correction in the plan

In section 3, the sentence "It does filter examples out before the baseline loss" conflicts with the rest of the plan. It should read: **"It does not filter examples out before the baseline loss."**

Validation informs evaluation, checkpoint selection, and stopping. Training examples remain eligible for learning even when their current errors already meet the candidate tolerance.

Source reviewed: `transmodel_training_plan.md`, especially sections 3 and 4. This document preserves the discussion and does not modify the training plan. No training or runtime measurements were performed for this review.